RED & WHITE SINCE2008
SKILL EDUCATION
SHAPING SKILLS FOR SCALING HIGHER...!!!

Project Title: "Smart Expense Tracker Application"

Objective

Build a Python-based **Smart Expense Tracker Application** that helps users log and analyze their spending patterns. The project must integrate **Control Structures, Arrays, OOP, NumPy, Pandas with Datasets, and Matplotlib & Seaborn** to deliver a feature-rich solution.

Problem Statement

You are tasked with developing a **Smart Expense Tracker Application** to help users monitor their expenses, categorize them, and visualize their financial habits. The application should enable data logging, analysis, and visualization of spending trends over time.

Features and Functionalities

1. Expense Input and Validation (Control Structure & Arrays - Low Weightage)

- Accept user input for expenses, including the date, amount, category, and description.
- Validate inputs using control structures (e.g., ensure the amount is positive and the category is valid).

2. Expense Tracker Class (OOP - Medium Weightage)

- Design an `ExpenseTracker` class with the following methods:
  - `add_expense(self, date, amount, category, description)`: Logs new expenses into the dataset.
  - `get_summary(self)`: Provides a summary of total and average expenses.
  - `filter_expenses(self, condition)`: Filters expenses by category, date range, or amount.
  - `generate_report(self)`: Outputs a summary report with key metrics.

3. Expense Analysis and Computations (NumPy & Pandas - Medium Weightage)

- Use **NumPy** for numerical calculations, such as average spending and total expenses per category.
- Use **Pandas** to:
    - Load, clean, and manage expense data.
    - Group expenses by category or month.
    - Identify top spending categories and periods.

4. Data Visualization (Matplotlib & Seaborn - High Weightage)

- Provide detailed financial visualizations:
    - **Bar Chart**: Total expenses by category.
    - **Line Graph**: Spending trends over time.
    - **Pie Chart**: Proportional spending distribution by category.
    - **Histogram**: Frequency of expense amounts.

Dataset Details

Use(by generating with the help of AI) a sample CSV file named `expenses.csv`, containing the following columns:

- `Date` (YYYY-MM-DD)
- `Amount`
- `Category` (e.g., Food, Transport, Utilities, Entertainment)
- `Description`

Implementation Guidelines

1. **Control Structures & Arrays**
    - Use loops and conditionals to validate user inputs and handle invalid data gracefully.
2. **OOP Design**
    - Structure the application using the `ExpenseTracker` class, with methods for each core functionality.
3. **NumPy Integration**
    - Perform computations like averages, percentages, and totals using NumPy arrays.
4. **Pandas for Data Handling**

- Use Pandas to manipulate, filter, and aggregate data efficiently.
- Create new metrics, such as "Monthly Average Spending" or "Spending Percentage by Category."

5. **Data Visualization**
   - Use Matplotlib and Seaborn to create visually appealing and informative graphs.
   - Ensure all graphs have clear labels, legends, and titles.

Submission Requirements

- Submit a Python script named `expense_tracker.py`.
- Attach the `expenses.csv` dataset used for testing.
- Include a `README.md` file with setup instructions and a description of the project.

Example Workflow

1. **Expense Logging**:
   - Allow users to input new expenses.
   - Validate data and store it in the dataset.
2. **Data Loading and Cleaning**:
   - Load existing data from `expenses.csv` into a Pandas DataFrame.
   - Handle missing or invalid data entries.
3. **Analysis and Metrics**:
   - Calculate the total, average, and category-wise spending using `get_summary()`.
4. **Data Filtering**:
   - Enable users to filter data by category or date range using `filter_expenses()`.
5. **Visualization**:
   - Generate bar charts, line graphs, pie charts, and histograms to highlight spending trends.

Key Notes for Students

- Focus on creating reusable, modular, and well-documented code.
- Pay attention to edge cases, such as invalid inputs or empty datasets.

- Ensure visualizations are clear, relevant, and professionally formatted.

Instructions:

1. Write clean, modular, and well-commented code.
2. Display meaningful outputs and prompts for user interaction.
3. Demonstrate OOP concepts and ensure all functionalities are implemented.
4. Submit your public GitHub repository containing a Python file (.py) and other necessary files (such as datasets if any)